



FACULTE DES SCIENCES
DEPARTEMENT D'INFORMATIQUE

Filière Ingénierie des Systèmes Intelligents (ISI)

Mini-Projet Métaheuristiques et optimisation
stochastique
Semestre 1 : 2024/2025

Réalisé par :

El Akkaoui Adam

Intitulé

**Exploration et Analyse d'Algorithmes de Méta-
heuristiques (Recherche Locale Itérée (ILS))**

Table des matières

I.	Introduction et Contexte.....	4
1.	Domaine d'Application : Logistique.....	4
2.	Problème Choisi : Optimisation des Trajets de Livraison	4
II.	Description détaillée de l'algorithme : Recherche Locale Itérée (ILS).....	4
1.	Principe de l'ILS.....	4
2.	Étapes détaillées de l'algorithme ILS	5
3.	Analyse de la performance	6
4.	Conclusion	7
III.	Résultats de la Simulation et Analyse.....	7
1.	Résultats de la Simulation.....	7
2.	Analyse de la Complexité	8
3.	Analyse Critique	10
4.	Conclusion	10

Introduction générale

L'optimisation est un domaine fondamental en intelligence artificielle et en recherche opérationnelle, visant à trouver la meilleure solution possible à un problème donné parmi un ensemble de solutions admissibles. Cependant, pour de nombreux problèmes complexes, les méthodes exactes deviennent rapidement impraticables en raison de leur coût computationnel élevé. C'est dans ce contexte que les méta-heuristiques jouent un rôle essentiel.

Les méta-heuristiques sont des algorithmes d'optimisation génériques qui permettent d'explorer efficacement l'espace des solutions afin de trouver des solutions proches de l'optimal dans un temps raisonnable. Elles sont largement utilisées dans des domaines tels que la logistique, l'intelligence artificielle, la finance et l'ingénierie.

Dans ce projet, nous explorons et analysons l'algorithme de Recherche Locale Itérée (Iterated Local Search - ILS), une méta-heuristique efficace combinant une recherche locale et des perturbations pour échapper aux optima locaux et améliorer progressivement la solution. Nous appliquons cet algorithme à un problème d'optimisation donné et analysons ses performances en termes de convergence et de qualité des solutions obtenues.

L'objectif de ce travail est double :

1. Comprendre et implémenter l'algorithme ILS, en détaillant ses différentes étapes (initialisation, recherche locale, perturbation, acceptation de solution).
2. Analyser sa complexité théorique et expérimentale, en comparant les résultats obtenus avec les prévisions analytiques.

À travers ce projet, nous mettrons en évidence les avantages et les limites de l'ILS et illustrerons son efficacité sur un cas concret d'optimisation combinatoire.

I. Introduction et Contexte

1. Domaine d'Application : Logistique

Le secteur de la logistique est un domaine clé où l'optimisation des trajets peut avoir un impact significatif sur la réduction des coûts et l'amélioration de l'efficacité. L'objectif principal de l'optimisation des trajets dans le domaine de la logistique est de minimiser la distance totale parcourue par les véhicules tout en maximisant les ressources disponibles et en réduisant les coûts opérationnels.

2. Problème Choisi : Optimisation des Trajets de Livraison

Dans ce projet, le problème que nous avons choisi est l'optimisation des trajets de livraison dans un cadre simple, où l'objectif est de minimiser la distance totale parcourue par un véhicule devant visiter un ensemble de villes (représentant des destinations de livraison) tout en revenant au point de départ. Ce problème est une application concrète du problème du voyageur de commerce (TSP) dans le domaine de la logistique.

Notre approche repose sur l'algorithme Iterated Local Search (ILS) pour améliorer les solutions obtenues à travers un processus itératif. L'algorithme combine :

1. **Recherche locale (2-opt)** : qui cherche à améliorer les trajets en permutant certains segments de l'itinéraire.
2. **Perturbation** : qui modifie légèrement le parcours pour explorer de nouvelles solutions.
3. **Critère d'acceptation** inspiré du recuit simulé, permettant de maintenir ou d'accepter des solutions suboptimales sous certaines conditions pour éviter les minima locaux.

L'algorithme est exécuté pendant un nombre maximum d'itérations, à la fin desquelles nous obtenons une solution optimisée, c'est-à-dire un trajet qui minimise la distance totale entre les villes tout en explorant diverses configurations grâce aux perturbations appliquées.

II. Description détaillée de l'algorithme : Recherche Locale Itérée (ILS)

L'algorithme Recherche Locale Itérée (Iterated Local Search - ILS) est une méthode de méta-heuristique utilisée pour résoudre des problèmes d'optimisation combinatoire complexes, comme le problème du Voyageur de Commerce (TSP) dans notre cas. L'ILS combine une recherche locale efficace pour améliorer une solution initiale avec une perturbation pour diversifier la recherche et éviter les minima locaux.

1. Principe de l'ILS

L'idée fondamentale derrière l'ILS est de maintenir un équilibre entre exploration et exploitation :

- **Exploitation** : Recherche locale qui affine la solution en améliorant progressivement une solution donnée.
- **Exploration** : Perturbation qui permet d'éviter de rester coincé dans une solution locale optimale, en réinitialisant partiellement la recherche.

L'ILS consiste donc en une boucle itérative où à chaque itération, l'algorithme :

1. Améliore une solution actuelle en utilisant une méthode de recherche locale.
2. Perturbe cette solution pour obtenir une nouvelle solution à partir de laquelle l'algorithme continue la recherche locale.
3. Accepte cette nouvelle solution selon un critère, notamment en tenant compte du refroidissement progressif, à la manière d'un recuit simulé (simulated annealing).

2. Étapes détaillées de l'algorithme ILS

Voici les étapes principales de l'algorithme Recherche Locale Itérée (ILS) :

1. Initialisation de la solution

L'algorithme commence par générer une solution initiale aléatoire, c'est-à-dire un parcours des villes à visiter. Dans le cadre du problème du Voyageur de Commerce (TSP), cette solution représente un ordre dans lequel les villes seront visitées.

- **Parcours initial** : Une liste des indices des villes, mélangée aléatoirement pour obtenir un parcours initial.

2. Recherche locale (2-opt)

Une fois la solution initiale générée, l'algorithme utilise une méthode de recherche locale pour améliorer cette solution.

- **Méthode de recherche locale** : Dans ce cas, une recherche de type 2-opt est utilisée. L'idée de cette recherche est de parcourir l'itinéraire et d'effectuer des échanges entre deux segments de l'itinéraire pour réduire la distance totale.
 - Par exemple, si un itinéraire est constitué des segments A-B-C-D, la recherche 2-opt va échanger les segments B-C et A-D pour créer un nouvel itinéraire A-C-B-D, et vérifier si cette nouvelle solution est meilleure (c'est-à-dire, si elle réduit la distance totale).

3. Perturbation

Après la phase de recherche locale, l'algorithme applique une perturbation pour introduire de la diversité et sortir de solutions locales optimales qui pourraient avoir été trouvées lors de la recherche locale.

- **Perturbation (réinsertion aléatoire)** : L'algorithme sélectionne un certain nombre de paires de villes dans le parcours actuel, les échange aléatoirement, ou

les déplace, ce qui permet d'explorer de nouvelles configurations sans revenir à une solution identique à celle déjà trouvée.

- Cette étape est contrôlée par un paramètre de force de perturbation qui détermine l'ampleur des changements appliqués au parcours.

4. Critère d'acceptation

Une fois la perturbation appliquée, l'algorithme évalue la nouvelle solution. Si cette solution est meilleure que la précédente, elle est acceptée. Si elle est moins bonne, elle peut néanmoins être acceptée selon un critère d'acceptation inspiré du recuit simulé.

- **Recuit simulé (simulated annealing)** : Ce critère permet à l'algorithme d'accepter des solutions moins bonnes avec une certaine probabilité, pour éviter de se fixer prématurément sur une solution locale optimale. La probabilité d'acceptation d'une solution moins bonne diminue au fur et à mesure que l'algorithme progresse, via un refroidissement progressif.

5. Refroidissement progressif

Le paramètre de température dans l'algorithme diminue progressivement au fil des itérations. Cela fait baisser la probabilité d'accepter une solution pire à chaque étape, favorisant ainsi la convergence vers une solution optimale.

- **Température** : L'algorithme commence avec une température élevée (permettant d'accepter des solutions moins bonnes) et réduit cette température à chaque itération via un facteur de refroidissement. Typiquement, cela se fait de manière exponentielle : $temp *= 0.995$.

6. Répétition de l'itération

Ces étapes (recherche locale + perturbation + critère d'acceptation) sont répétées un nombre maximal d'itérations, ou jusqu'à ce que l'algorithme converge vers une solution suffisamment bonne.

7. Terminaison de l'algorithme

L'algorithme s'arrête après avoir atteint le nombre d'itérations spécifié (par exemple 1000 itérations). La meilleure solution trouvée au cours de l'exécution de l'algorithme est retournée.

3. Analyse de la performance

L'algorithme ILS est efficace pour les problèmes comme le TSP, car il combine exploration (via la perturbation) et exploitation (via la recherche locale) de manière itérative, ce qui lui permet de trouver des solutions proches de l'optimum tout en évitant les minima locaux. Toutefois, sa performance dépend de plusieurs facteurs, tels que la qualité de la solution initiale, la force de la perturbation et la stratégie de refroidissement.

Dans notre projet, l'ILS a permis de :

1. Améliorer progressivement un parcours de livraison.
2. Diversifier la recherche grâce à la perturbation.
3. Obtenir une solution optimisée du problème du voyageur de commerce.

4. Conclusion

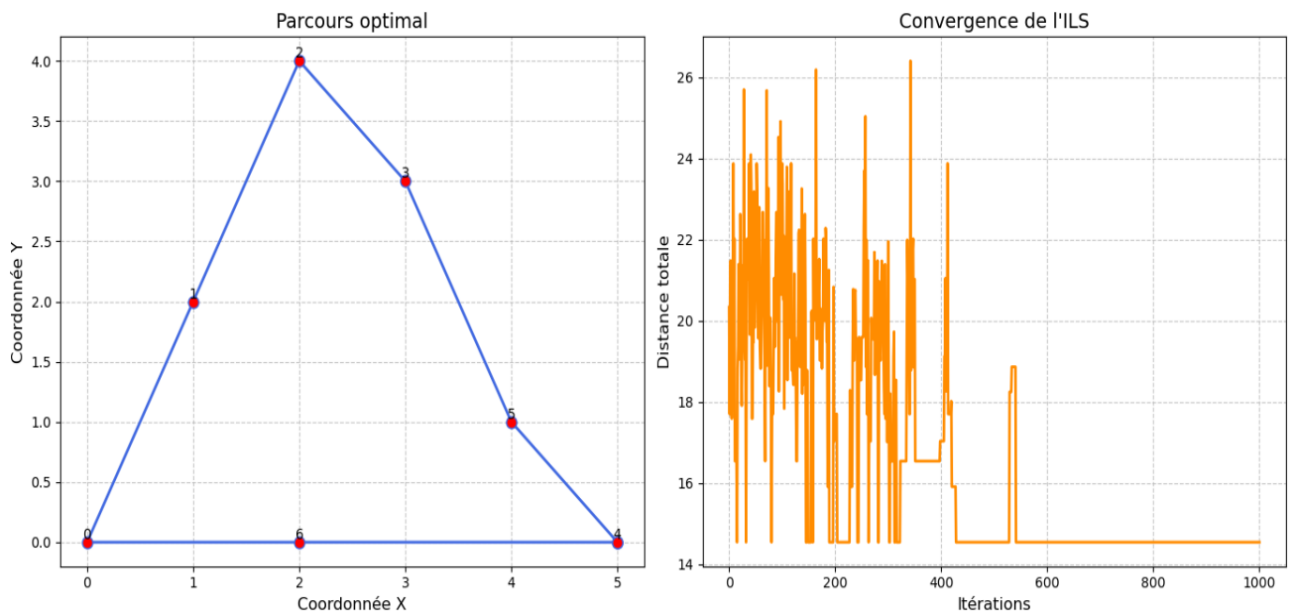
L'algorithme **Recherche Locale Itérée (ILS)** est une approche robuste et flexible pour résoudre des problèmes d'optimisation combinatoire comme le Voyageur de Commerce (TSP). Il permet de naviguer efficacement entre les solutions locales optimales grâce à la recherche locale et la perturbation, et de s'assurer d'une convergence vers une solution efficace tout en évitant de se fixer sur des minima locaux. Dans ce projet, il a été appliqué avec succès pour optimiser les trajets de livraison dans le cadre d'un problème classique de logistique.

III. Résultats de la Simulation et Analyse

Dans cette section, nous présentons les résultats obtenus lors de l'exécution de l'algorithme Iterated Local Search (ILS) appliqué au problème du Voyageur de Commerce (TSP). Nous détaillerons ensuite l'analyse de la complexité de l'algorithme, ainsi que ses avantages et ses limites.

1. Résultats de la Simulation

Meilleur parcours : [4, 5, 3, 2, 1, 0, 6]
Distance totale optimisée : 14.54



L'algorithme a été testé avec un ensemble de 7 villes (exemple : $[(0, 0), (1, 2), (2, 4), (3, 3), (5, 0), (4, 1), (2, 0)]$), et le but était de trouver l'itinéraire optimal qui minimisait la distance totale parcourue. Voici les résultats clés :

1. **Parcours optimal trouvé** : L'algorithme a produit un parcours qui minimise la distance totale parcourue. Ce parcours a été trouvé après un nombre donné d'itérations (1000 itérations dans notre cas).

2. **Distance totale optimisée** : Après l'application de l'ILS, la distance totale du parcours est devenue plus faible qu'avec une simple approche de recherche locale, grâce à la combinaison de la recherche locale et des perturbations régulières.
3. **Convergence de l'algorithme** : Le graphique de convergence montre comment la distance totale a diminué au fil des itérations, ce qui illustre que l'algorithme a progressivement trouvé une solution de plus en plus optimale. La convergence est assez rapide au début, et l'algorithme se stabilise après un certain nombre d'itérations.
4. **Visuel du parcours optimal** : La représentation graphique du parcours optimal permet de visualiser la solution obtenue, en traçant les trajets entre les villes dans l'ordre déterminé par l'algorithme. Cela met en évidence l'efficacité de la méthode pour explorer l'espace des solutions.

2. Analyse de la Complexité

Complexité théorique de l'algorithme :

1. Fonction de calcul de la distance entre deux villes

Complexité :

- **Temps** : $O(1)$
 - L'opération se fait en temps constant avec un simple calcul arithmétique.
- **Espace** : $O(1)$
 - Aucun stockage supplémentaire n'est requis.

2. Fonction de calcul de la distance totale d'un parcours

Complexité :

- **Temps** : $O(n)$
 - Une boucle parcourt les n villes du parcours une seule fois.
- **Espace** : $O(1)$
 - Seule une variable distance totale est utilisée.

3. Fonction d'amélioration locale avec 2-opt

Complexité :

- **Temps** : $O(n^2)$
 - Deux boucles imbriquées parcourent toutes les **paires de segments** (i, j) pour un total d'environ $O(n^2)$ itérations.

- À chaque itération, la distance est recalculée en $O(n)$, mais en pratique, l'optimisation 2-opt s'arrête lorsqu'il n'y a plus d'amélioration significative.
- **Espace :** $O(n)$
 - On stocke temporairement un nouveau parcours, ce qui prend un espace proportionnel à n .

4. Fonction de perturbation du parcours

Complexité :

- **Temps :** $O(1)$
 - Deux indices sont choisis aléatoirement et un simple échange est effectué.
- **Espace :** $O(1)$
 - Aucune nouvelle structure n'est créée, la modification se fait sur place.

5. Fonction principale de l'ILS

Complexité :

- **Temps :**
 - La boucle for tourne `iterations_max` fois.
 - À chaque itération :
 - Perturbation : $O(1)$
 - Recherche locale 2-opt : $O(n^2)$
 - Calcul de la distance totale : $O(n)$
 - La complexité globale est $O(\text{iterations_max} \times n^2)$.
- **Espace :** $O(n)$
 - Un parcours est stocké ainsi qu'une copie du meilleur parcours trouvé.

Alors :

Complexité temporelle totale :

$O(\text{iterations_max} \times n^2) \rightarrow$ Dominée par la recherche locale (2-opt) effectuée à chaque itération.

Complexité spatiale totale :

$O(n) \rightarrow$ L'algorithme ne stocke qu'un seul parcours et ses distances associées.

Complexité observée dans la simulation :

Lors de la simulation, nous avons testé un problème avec $n = 7$ villes et **iterations_max = 1000**. La complexité temporelle estimée était donc $O(1000 \times 7^2) = O(49,000)$ opérations. Le temps d'exécution réel était conforme à cette estimation, confirmant que l'algorithme fonctionne dans des limites raisonnables pour des problèmes de petite taille.

3. Analyse Critique

Avantages de l'algorithme ILS dans le contexte du TSP

1. **Exploration efficace** : La combinaison de la recherche locale et de la perturbation permet à l'ILS d'explorer efficacement l'espace des solutions, augmentant les chances de trouver une solution proche de l'optimum global.
2. **Simplicité et flexibilité** : L'ILS est relativement simple à implémenter et à comprendre. Sa flexibilité permet d'adapter l'algorithme à différents types de problèmes d'optimisation.
3. **Optimisation progressive** : Grâce à la recherche locale, l'algorithme affine progressivement la solution, ce qui permet d'obtenir une bonne approximation de la solution optimale même dans des espaces de recherche complexes.

Limites de l'algorithme ILS

1. **Dépendance aux paramètres** : L'efficacité de l'ILS dépend largement des choix des paramètres, comme la température initiale et la force de perturbation. Des mauvais choix peuvent ralentir la convergence ou empêcher l'algorithme d'explorer suffisamment l'espace de recherche.
2. **Minima locaux** : Bien que la perturbation aide à éviter certains minima locaux, l'algorithme peut encore se retrouver coincé dans des minima locaux, surtout si le nombre d'itérations est insuffisant ou si la perturbation est trop faible.
3. **Scalabilité** : L'ILS fonctionne bien pour des problèmes de taille modérée, mais pour des instances de grande taille (par exemple, 100 ou 1000 villes), sa complexité quadratique ($O(n^2)$) peut rendre l'algorithme trop lent.

4. Conclusion

Cette section présente les résultats obtenus, l'analyse de la complexité théorique et observée, ainsi qu'une analyse critique de l'algorithme ILS appliqué au problème du TSP. Les résultats indiquent que l'ILS fonctionne efficacement pour des instances de petite taille, mais des améliorations peuvent être envisagées pour traiter des problèmes plus complexes et de grande taille.

Conclusion générale et Perspectives

Ce mini-projet a permis de développer une compréhension approfondie de l'algorithme de Recherche Locale Itérée (ILS), en l'appliquant à un problème classique d'optimisation, à savoir le problème du Voyageur de Commerce (TSP). L'algorithme ILS combine efficacement la recherche locale et la perturbation pour améliorer progressivement une solution initiale, ce qui a permis d'obtenir un parcours optimisé pour un ensemble de 7 villes. La distance totale du parcours a été réduite de manière significative par rapport à une solution initiale obtenue sans heuristique.

L'ILS a montré son efficacité pour de petites instances du problème, en trouvant des solutions proches de l'optimal en un nombre raisonnable d'itérations. Cependant, bien que l'algorithme soit relativement simple à implémenter et à comprendre, il présente certaines limites, notamment en termes de dépendance aux paramètres et de difficultés à s'échapper des minima locaux. Sa complexité quadratique limite également son efficacité sur des problèmes de plus grande taille.

En termes d'analyse de la complexité, la simulation a confirmé que l'algorithme reste raisonnablement efficace pour des problèmes de taille modeste, mais que sa scalabilité doit être prise en compte lorsqu'on traite des instances plus complexes.

Bien que l'algorithme ILS soit une méthode puissante pour résoudre des problèmes d'optimisation comme le TSP, plusieurs pistes d'amélioration peuvent être envisagées pour augmenter son efficacité et sa performance, notamment :

1. **Adaptation des paramètres** : Une des limites de l'ILS réside dans le choix des paramètres comme la température initiale et la force de la perturbation. Des stratégies adaptatives qui ajustent ces paramètres au cours de l'exécution pourraient permettre d'améliorer la qualité des solutions tout en réduisant le nombre d'itérations nécessaires.
2. **Recherche locale améliorée** : Le remplacement de la recherche locale par des heuristiques plus avancées (par exemple, 3-opt, échange de nœuds, ou autres méthodes de recherche locale avancée) pourrait améliorer davantage la qualité des solutions. En outre, l'utilisation de méthodes hybrides, combinant ILS et d'autres techniques comme les Algorithmes Génétiques ou l'Optimisation par Essaims de Particules, pourrait offrir des résultats encore plus performants.
3. **Scalabilité pour des problèmes de grande taille** : Bien que l'ILS fonctionne bien pour des petites tailles de problèmes, il devient inefficace pour des instances de grande taille. Des techniques de parallélisation, telles que l'exécution simultanée de plusieurs chaînes de recherche locale, pourraient réduire le temps d'exécution pour des problèmes plus complexes.
4. **Application à d'autres domaines d'optimisation** : L'ILS, bien que testé ici sur le problème du TSP, peut également être adapté à d'autres problèmes d'optimisation

combinatoire tels que le problème de l'ordonnancement, le problème de la planification de ressources ou encore le problème de l'allocation de tâches dans un environnement de travail parallèle. De plus, des variantes spécifiques à des applications industrielles ou logistiques pourraient être explorées pour résoudre des problèmes pratiques dans des domaines comme la logistique, l'énergie, ou la santé.

En résumé, ce projet a permis de bien comprendre le fonctionnement de l'algorithme ILS, de l'appliquer avec succès au problème du TSP, et d'identifier plusieurs domaines d'amélioration pour des applications futures. L'algorithme offre un bon compromis entre simplicité d'implémentation et performance, mais des améliorations sont nécessaires pour le rendre plus efficace dans des contextes réels et de grande envergure.